Banking Management System

Use Case Model

CS487 'A' — Object-Oriented Programming

Group: Aarogya, Parth, Sein, Cho
Professor: Dr. Shak Hossain
Date: March 10, 2026

Part 1: Actor Identification

Primary Actors

Primary actors are human users who directly initiate interactions with the Banking Management System to accomplish specific goals.

Customer: An individual who holds one or more bank accounts. The customer uses the system to perform everyday banking activities including depositing and withdrawing funds, transferring money between accounts, viewing transaction history, applying for loans, tracking loan balances, and processing card-based transactions. The customer must authenticate before accessing any functionality.

Admin (Bank Administrator): A bank employee with elevated privileges who manages the operational aspects of the system. The admin is responsible for registering new customers, creating accounts, reviewing and approving or rejecting loan applications, issuing debit and credit cards, and updating customer information. The admin also authenticates through the same login system but gains access to administrative features.

Secondary Actors

Secondary actors are external systems or services that support or are invoked by the Banking Management System during the execution of use cases, but do not initiate interactions themselves.

Authentication Service: An internal module or external identity service that verifies user credentials (username and password) and determines the role (Customer or Admin) to enforce role-based access control throughout the system.

Transaction Logging Service: A backend service responsible for generating unique transaction IDs, recording timestamps, categorizing transaction types (deposit, withdrawal, transfer), and maintaining a persistent transaction history for each account.

Interest Calculation Engine: A system component that periodically computes and applies interest to eligible savings accounts based on the account balance and the applicable interest rate, without requiring manual initiation by any human actor.

Notification Service: A supporting service that sends confirmations and alerts to customers, such as loan approval/rejection notifications, card issuance confirmations, and large-transaction alerts.

Part 2: Use Case Identification

The table below lists all major use cases identified for the Banking Management System, organized by functional category.

Use Case Name	Brief Description	Primary Actor	Category
Log In	User authenticates with credentials (username and password) to access role-based features.	Customer / Admin	System
Register Customer	Admin creates a new customer profile with personal details and links it to the system.	Admin	Admin
Update Customer Info	Admin or Customer updates personal information such as name, address, or contact details.	Customer / Admin	Customer
View Customer Details	Customer or Admin views the full profile and linked accounts of a customer.	Customer / Admin	Customer
Create Account	Customer or Admin opens a new savings or checking account linked to a customer profile.	Customer / Admin	Account
Deposit Funds	Customer deposits money into a specified account; system updates balance and logs the transaction.	Customer	Account
Withdraw Funds	Customer withdraws money from an account; system validates balance/overdraft and logs the transaction.	Customer	Account
Transfer Funds	Customer transfers money between two accounts; system validates both balances and logs two transactions.	Customer	Account
View Transaction History	Customer views a chronological list of all transactions for a given account.	Customer	Account
Apply for Loan	Customer submits a loan application specifying amount, term, and purpose.	Customer	Loan
Approve/Reject Loan	Admin reviews the loan application and approves or rejects it based on eligibility criteria.	Admin	Loan
Calculate EMI	System computes the Equated Monthly Installment for an approved loan based on amount, rate, and term.	System	Loan
Track Loan Balance	Customer views the remaining loan balance and repayment schedule.	Customer	Loan
Issue Card	System issues a debit or credit card linked to a customer's account.	Admin	Card

Validate PIN	System verifies the customer's PIN before authorizing a card-based transaction.	System	Card
Process Card Transaction	Customer uses a debit or credit card for a payment; system checks PIN, limit, and balance.	Customer	Card
Apply Interest	System calculates and applies periodic interest to eligible savings accounts.	System	System
Check Overdraft Eligibility	System verifies whether a checking account qualifies for overdraft during a withdrawal.	System	System

Part 3: Use Case Diagram

The UML Use Case Diagram for the Banking Management System is provided as an attached file. The diagram displays all actors positioned outside the system boundary and all use cases as labeled ellipses inside the system boundary rectangle.

Diagram Relationship Summary

The following relationships are modeled in the diagram:

«include» Relationships (mandatory sub-behavior)

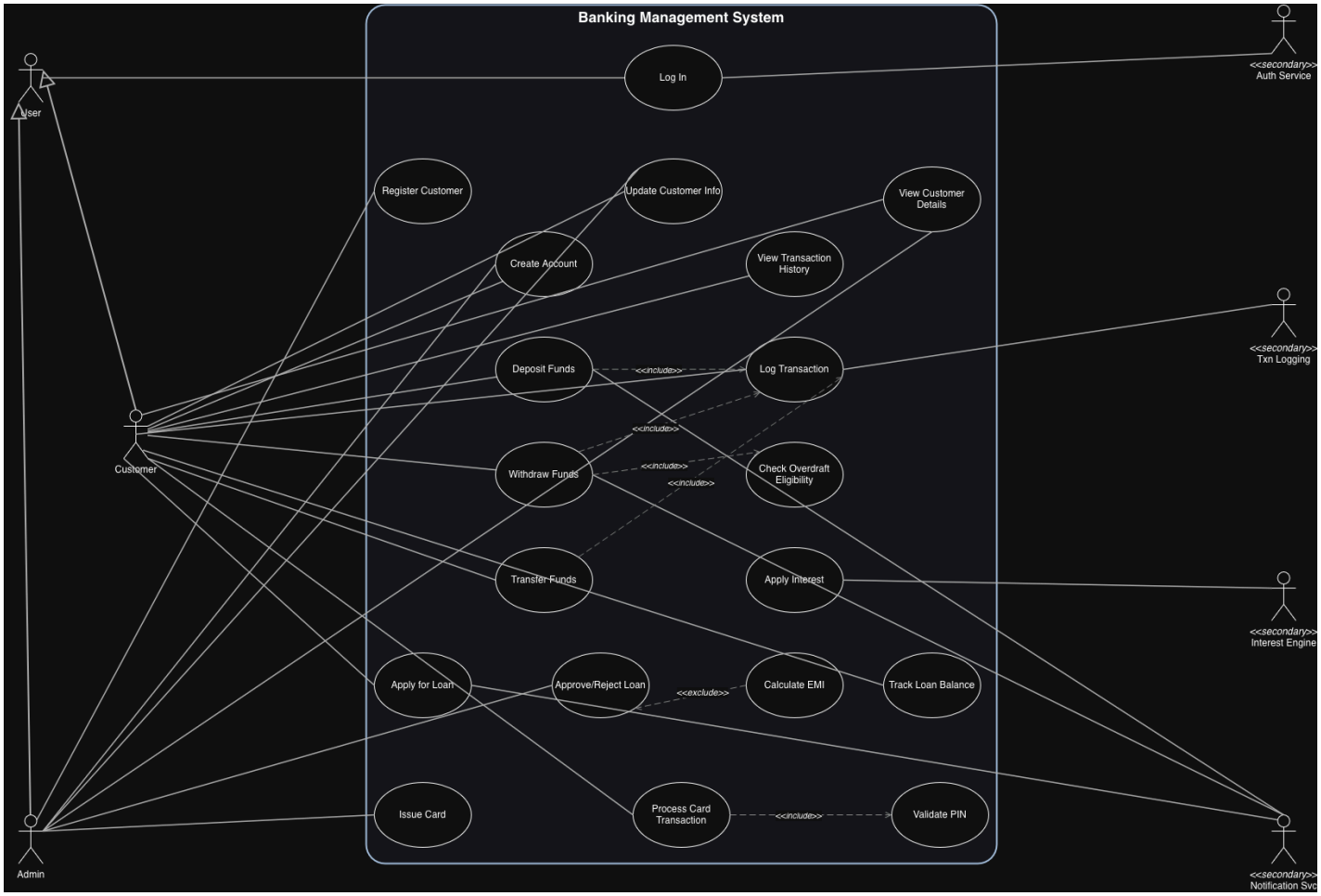
- "Deposit Funds" includes "Log Transaction" — every deposit must be recorded.
- "Withdraw Funds" includes "Log Transaction" — every withdrawal must be recorded.
- "Withdraw Funds" includes "Check Overdraft Eligibility" — system must verify overdraft status for checking accounts.
- "Transfer Funds" includes "Log Transaction" — both debit and credit legs are recorded.
- "Process Card Transaction" includes "Validate PIN" — PIN verification is mandatory before any card operation.

«extend» Relationships (conditional behavior)

- "Apply Interest" extends "Deposit Funds" — interest is conditionally applied during scheduled processing, not every deposit.
- "Calculate EMI" extends "Approve/Reject Loan" — EMI is calculated only when a loan is approved.

Generalization

- Customer and Admin both generalize to a common "User" actor for the "Log In" use case, since both roles authenticate through the same mechanism.


Banking Management System
User
Log In
<<secondary>>
Auth Service
Register Customer
Update Customer Info
View Customer Details
Create Account
View Transaction History
<<secondary>>
Txn Logging
Deposit Funds
<<include>>
Log Transaction
<<include>>
Customer
Withdraw Funds
<<include>>
<<include>>
Check Overdraft Eligibility
Transfer Funds
Apply Interest
<<secondary>>
Interest Engine
Apply for Loan
Approve/Reject Loan
<<exclude>>
Calculate EMI
Track Loan Balance
Issue Card
Process Card Transaction
<<include>>
Validate PIN
Admin
<<secondary>>
Notification Svc

Part 4: Detailed Use Case Specifications

Use Case 1: Withdraw Funds

Use Case Name: Withdraw Funds

Use Case ID: UC-07

Primary Actor: Customer

Secondary Actors: Transaction Logging Service, Notification Service

Scope: Banking Management System

Preconditions:

- The customer is logged in and authenticated.
- The customer has at least one active account (savings or checking).
- The account has a non-negative balance (or overdraft is enabled for checking accounts).

Postconditions (Success):

- The account balance is reduced by the withdrawal amount.
- A transaction record with a unique ID, timestamp, and type "Withdrawal" is created.
- The updated balance is displayed to the customer.

Main Success Scenario

1. The customer navigates to the Withdrawal page and selects the target account.
2. The system displays the current account balance.
3. The customer enters the withdrawal amount.
4. The system validates that the amount is a positive number.
5. The system checks whether the account has sufficient funds. [«include» Check Overdraft Eligibility for checking accounts]
6. The system deducts the amount from the account balance.
7. The system generates a unique transaction ID and records the transaction with a timestamp and type "Withdrawal." [«include» Log Transaction]
8. The system displays a success message with the new balance.
9. The system sends a confirmation notification to the customer.

Alternative Flows

A1: Overdraft Triggered (Checking Account Only)

- At step 5, the system detects that the withdrawal amount exceeds the available balance in a checking account.
- The system verifies that overdraft is enabled and the requested amount is within the overdraft limit.
- The system allows the withdrawal, deducting the amount and setting the balance to a negative value within the overdraft threshold.
- The transaction is logged with a note indicating overdraft was used.
- The flow resumes at step 8.

A2: Customer Cancels

- At step 3, the customer clicks "Cancel" before confirming the amount.
- The system discards the operation and returns the customer to the account dashboard.
- No changes are made to the account balance or transaction history.

Exceptions

E1: Insufficient Funds (Savings Account)

- At step 5, the system determines that the savings account balance is less than the requested withdrawal amount.
- The system displays an error message: "Insufficient funds. Your current balance is $X.XX."
- The withdrawal is denied; the customer is returned to the withdrawal page.

E2: Overdraft Limit Exceeded (Checking Account)

- At step 5, the overdraft check determines that the withdrawal would exceed the allowed overdraft limit.
- The system displays an error message indicating the maximum withdrawal amount allowed.
- The withdrawal is denied.

E3: Invalid Amount

- At step 4, the system detects a non-numeric, zero, or negative input.
- The system displays a validation error and prompts the customer to enter a valid positive amount.

E4: System/Database Error

- If the system encounters a database or network error at any point, it displays a generic error message, does not alter the account balance, and logs the error for administrators.

Use Case 2: Apply for Loan

Use Case Name: Apply for Loan

Use Case ID: UC-10

Primary Actor: Customer

Secondary Actors: Notification Service

Scope: Banking Management System

Preconditions:

- The customer is logged in and authenticated.
- The customer has at least one active account.
- The customer does not have an existing loan application in "Pending" status.

Postconditions (Success):

- A new loan application record is created with a unique loan ID and status "Pending."
- The application details (amount, term, purpose) are stored in the system.
- The customer receives a confirmation notification that the application has been submitted.

Main Success Scenario

1. The customer navigates to the Loan Application page.
2. The system displays the loan application form with fields for loan amount, repayment term (in months), and loan purpose.
3. The customer enters the desired loan amount.
4. The customer selects a repayment term from available options (e.g., 12, 24, 36, 48, or 60 months).
5. The customer enters a brief description of the loan purpose.
6. The customer reviews the entered information and clicks "Submit Application."
7. The system validates all input fields (amount is positive, term is selected, purpose is not empty).
8. The system generates a unique loan ID and creates a loan record with status "Pending."
9. The system displays a confirmation message: "Your loan application (ID: LXXXX) has been submitted and is awaiting review."
10. The system sends a submission confirmation notification to the customer.

Alternative Flows

A1: Customer Saves Draft

- At step 6, the customer clicks "Save Draft" instead of "Submit Application."
- The system saves the partially completed application with status "Draft."
- The customer can return later to complete and submit the application.

A2: Customer Modifies Before Submission

- At step 6, the customer reviews and notices an error in the entered data.
- The customer edits the relevant field(s) directly on the form.
- The flow resumes at step 6.

Exceptions

E1: Invalid Loan Amount

- At step 7, the system detects that the loan amount is zero, negative, or exceeds the maximum allowed limit.
- The system displays a validation error indicating the acceptable range.
- The application is not submitted; the customer corrects the amount.

E2: Existing Pending Application

- At step 1, the system detects that the customer already has a loan application with status "Pending."
- The system displays a message: "You already have a pending loan application. Please wait for a decision before submitting a new one."

E3: Missing Required Fields

- At step 7, the system detects that one or more required fields are empty.
- The system highlights the missing fields and displays a prompt to complete them before submission.

E4: System/Database Error

- If the system encounters an error at any point, it displays a generic error, preserves the entered data in the form, and logs the issue for administrators.

Part 5: Reflection

How does use case design help before writing code?

Use case design is a critical first step in software development because it forces the team to systematically think through who will use the system and exactly what they need to accomplish. For our Banking Management System, writing use cases before coding helped us identify not just the obvious features like deposits and withdrawals, but also the less obvious system behaviors such as overdraft checking, transaction logging, and the conditional nature of interest application. Without this upfront analysis, we would likely have discovered these requirements mid-implementation, leading to major refactoring or inconsistent handling across modules.

Furthermore, use case specifications define the precise interaction sequences between actors and the system, including alternative paths and error handling. This gives every team member a shared, unambiguous understanding of expected behavior. For a group project like ours with four members, this shared blueprint is essential for dividing work without creating integration conflicts. Each developer can implement their assigned use cases knowing exactly what inputs to expect, what validations to perform, and what outputs to produce. Use cases also serve as a natural foundation for writing test cases—each step in the main flow and each exception becomes a testable scenario, which directly supports our objective of thorough testing and debugging.

How might use cases influence your class design in OOP?

Use cases have a direct and significant influence on how we design our classes in an object-oriented system. The actors identified in the use case model map naturally to a class hierarchy: both Customer and Admin generalize to a common User actor for login, which suggests a User base class with shared authentication attributes and methods, and specialized Customer and Admin subclasses that add role-specific functionality. This is a clear application of inheritance and polymorphism—the login() method can be defined in User, while methods like applyForLoan() belong exclusively to Customer and approveLoan() to Admin.

The use cases also reveal the entities and service classes needed. The "Deposit Funds," "Withdraw Funds," and "Transfer Funds" use cases all interact with Account objects and include transaction logging, suggesting an Account base class (with SavingsAccount and CheckingAccount subclasses that override withdrawal behavior for overdraft handling) and a Transaction class for recording each operation. The loan-related use cases indicate the need for a Loan class with attributes like loanID, amount, term, status, and methods like calculateEMI(). The card use cases suggest a Card base class with DebitCard and CreditCard subclasses that differ in limit handling and validation. Composition is evident everywhere—a Customer object has-a list of Account objects, each Account has-a list of Transaction objects, and so on. By mapping use case interactions to class responsibilities, we ensure that our code architecture is driven by actual user needs and system behaviors rather than arbitrary technical decisions, resulting in a design that is both modular and maintainable.